

Pemrograman Berorientasi Objek (PBO)

Pertemuan 02: Java Dasar 1

Dany Candra F, S.Kom,.,M.Eng

Review Pertemuan Sebelumnya

- . *Object Oriented Programming* adalah sudut pandang bahasa pemrograman yang berkonsep “objek”. Dengan tujuan membuat proses development menjadi lebih efisien dengan mengurangi kode yang **redudansi** dan membuat perangkat lunak yang **reusable**.
- . *Class* adalah *blueprint, prototype* atau cetakan untuk membuat Object berisikan deklarasi semua *properties* dan *functions* yang dimiliki oleh *Object*
- . *Object* adalah representasi dari class
- . Java adalah Bahasa pemrograman yang mendukung OOP.
- . JVM merupakan teknologi yang dapat membuat java berjalan dibanyak *platform*.
- . JVM ➔ JRE (dibutuhkan untuk menjalankan app java) ➔ JDK (dibutuhkan untuk develop dengan java)

Tujuan Pembelajaran Java Dasar 1

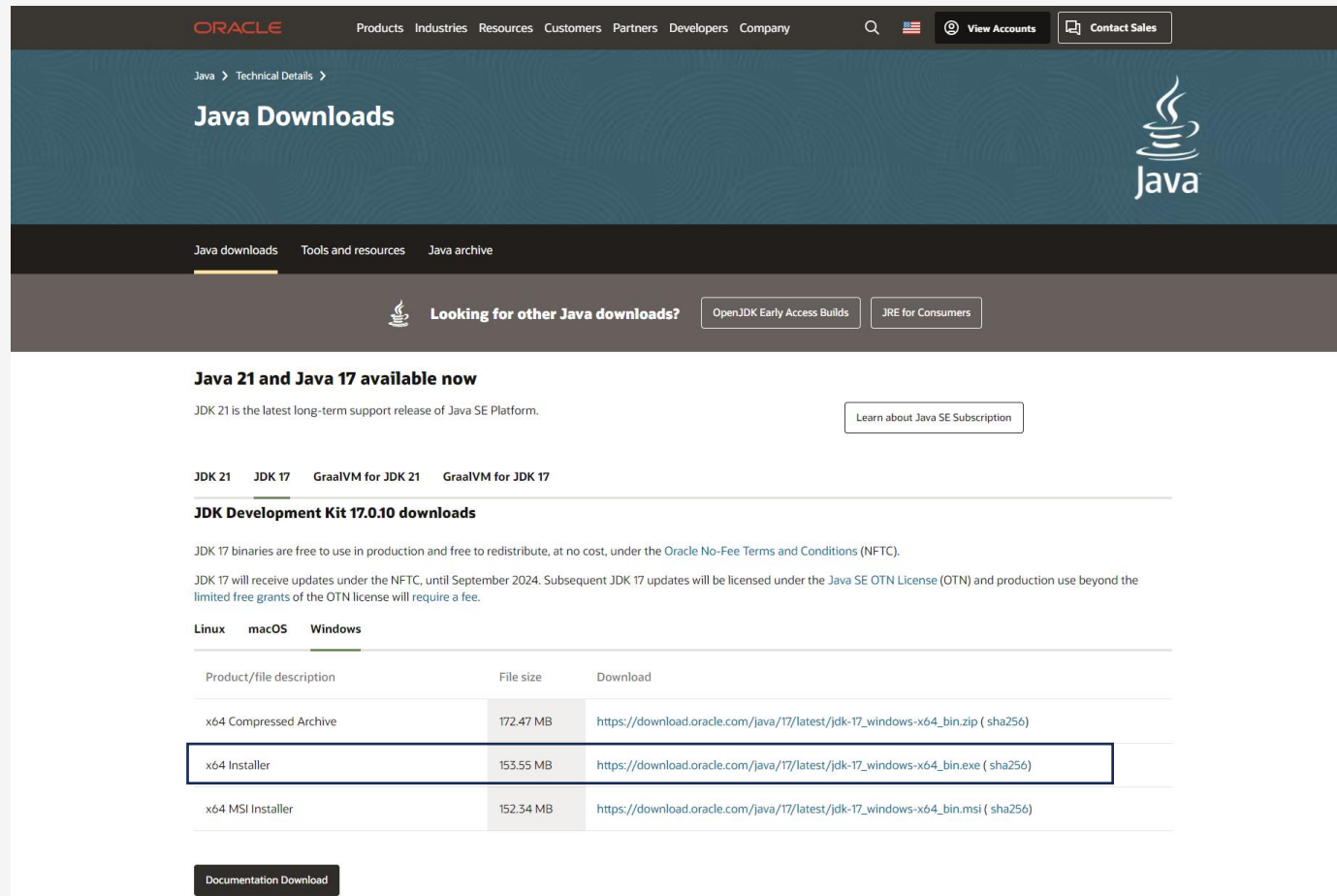
- . Mahasiswa dapat memahami teknologi Java
- . Mahasiswa dapat memahami dan menggunakan sintaks Java.

Agenda

- . Setup Environment (JDK & Netbeans IDE)
 - . Menulis sintaks java
 - . Tipe Data pada Java
 - . Variable
 - . Operator
 - . Input & Output
 - . Array
- 

Instalasi JDK : Download JDK

Buka Halaman [Java Downloads | Oracle](https://www.oracle.com/technetwork/java/javase-downloads-428756.html)



The screenshot shows the Oracle Java Downloads page. The header includes the Oracle logo, navigation links (Products, Industries, Resources, Customers, Partners, Developers, Company), a search icon, a US flag, and buttons for 'View Accounts' and 'Contact Sales'. The main content area has a dark blue header with 'Java Downloads' and the Java logo. Below this is a navigation bar with 'Java downloads', 'Tools and resources', and 'Java archive'. A section titled 'Looking for other Java downloads?' includes links for 'OpenJDK Early Access Builds' and 'JRE for Consumers'. The main content area features a section for 'Java 21 and Java 17 available now', stating that JDK 21 is the latest long-term support release. Below this is a 'JDK Development Kit 17.0.10 downloads' section, which includes a table of download links for Linux, macOS, and Windows. The 'x64 Installer' link is highlighted with a blue border. A 'Documentation Download' button is located at the bottom left.

Java 21 and Java 17 available now

JDK 21 is the latest long-term support release of Java SE Platform.

[Learn about Java SE Subscription](#)

JDK Development Kit 17.0.10 downloads

JDK 17 binaries are free to use in production and free to redistribute, at no cost, under the [Oracle No-Fee Terms and Conditions \(NFTC\)](#).

JDK 17 will receive updates under the NFTC, until September 2024. Subsequent JDK 17 updates will be licensed under the [Java SE OTN License \(OTN\)](#) and production use beyond the limited free grants of the OTN license will require a fee.

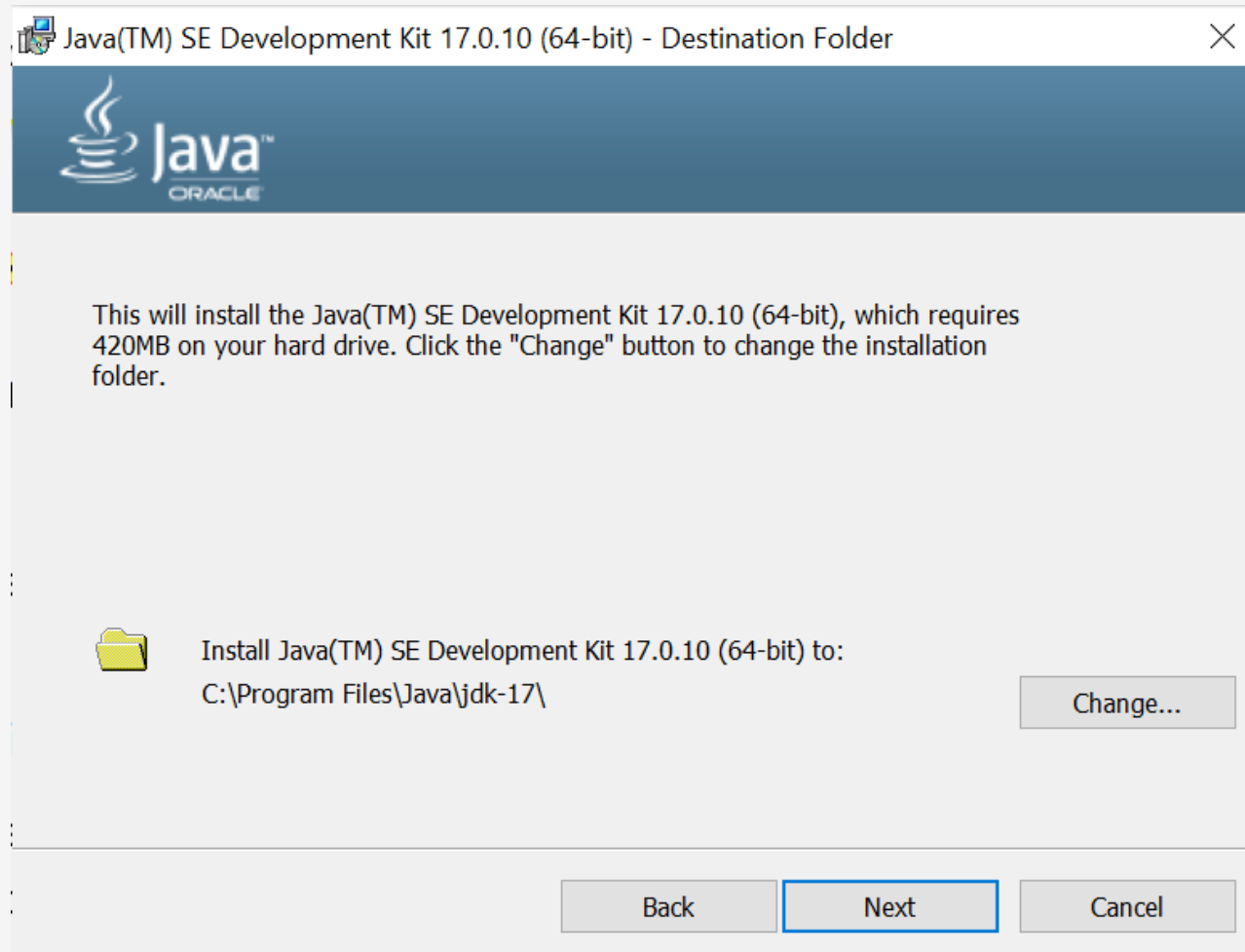
Linux macOS Windows

Product/file description	File size	Download
x64 Compressed Archive	172.47 MB	https://download.oracle.com/java/17/latest/jdk-17_windows-x64_bin.zip (sha256)
x64 Installer	153.55 MB	https://download.oracle.com/java/17/latest/jdk-17_windows-x64_bin.exe (sha256)
x64 MSI Installer	152.34 MB	https://download.oracle.com/java/17/latest/jdk-17_windows-x64_bin.msi (sha256)

[Documentation Download](#)

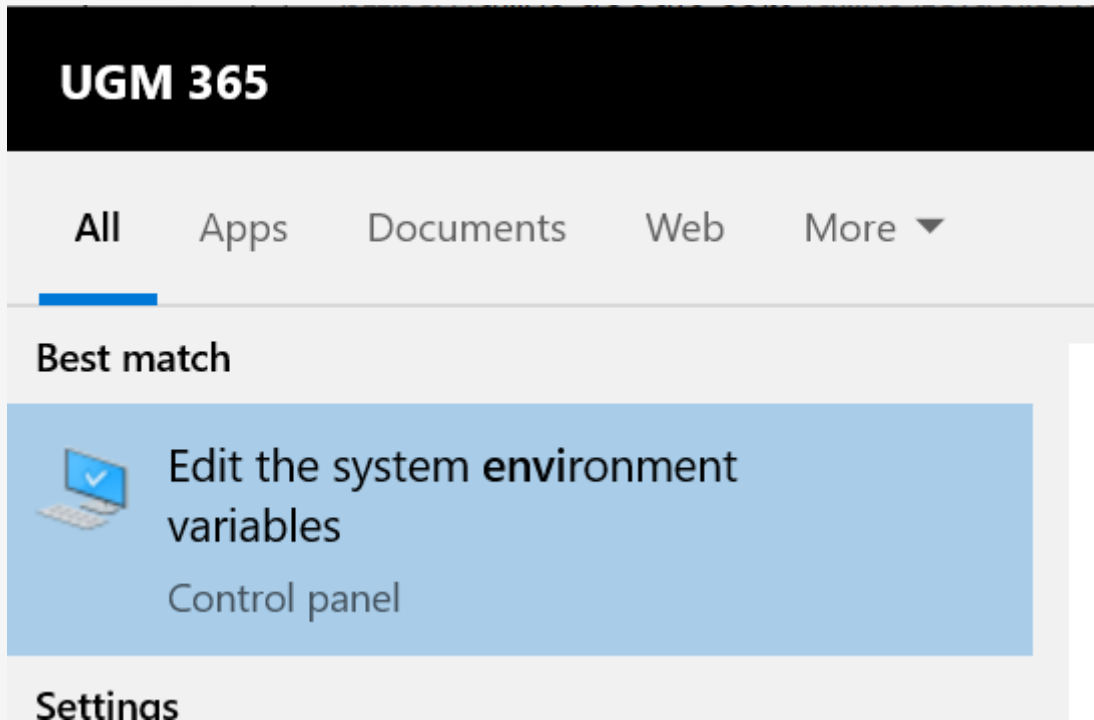
Instalasi JDK : Install JDK pada Device

Ikuti langkah instalasi

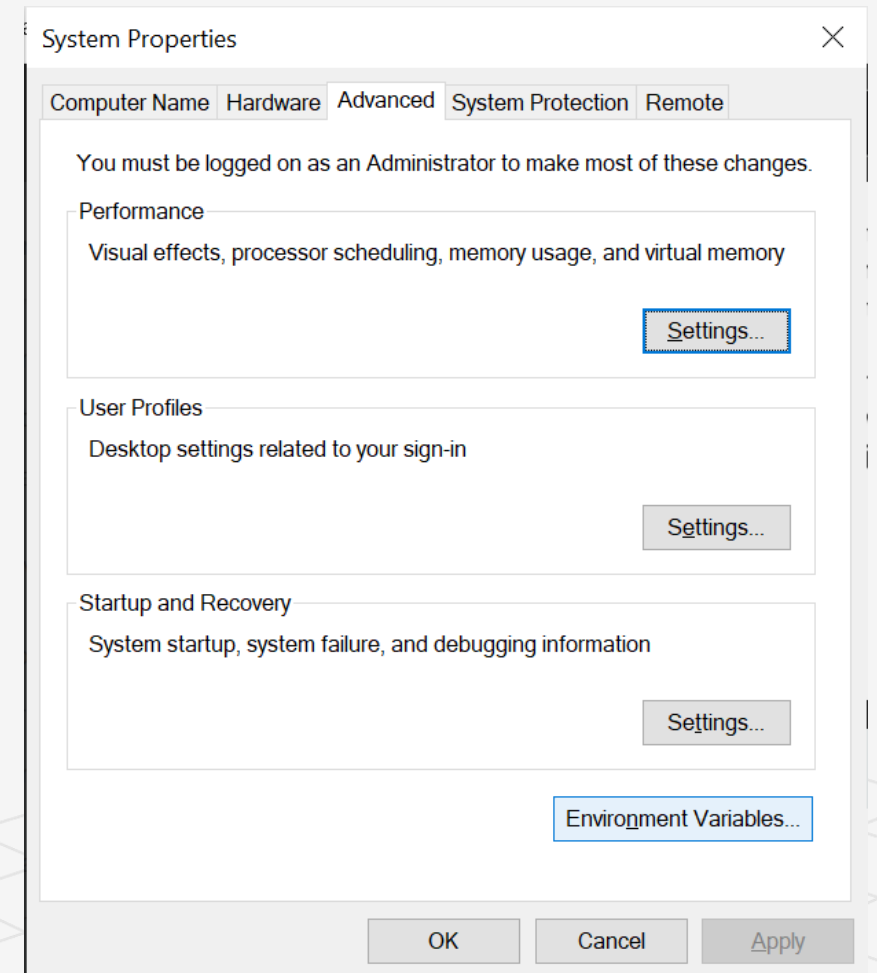


Instalasi JDK : Setting path JAVA pada windows

Pilih Environment Variable pada Windows



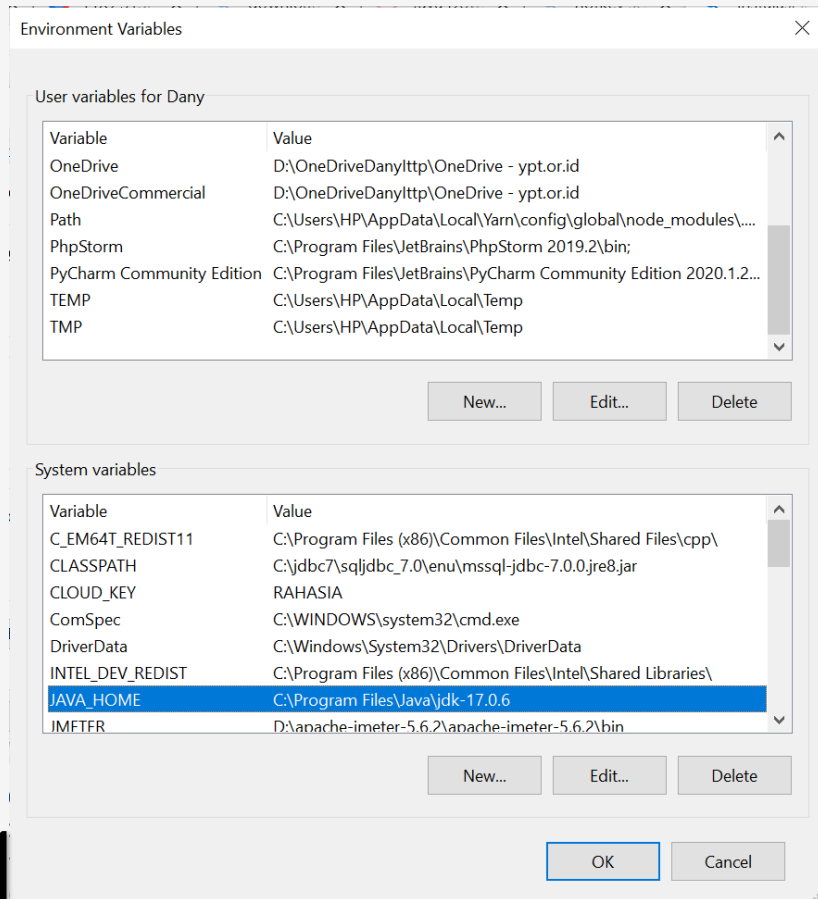
1



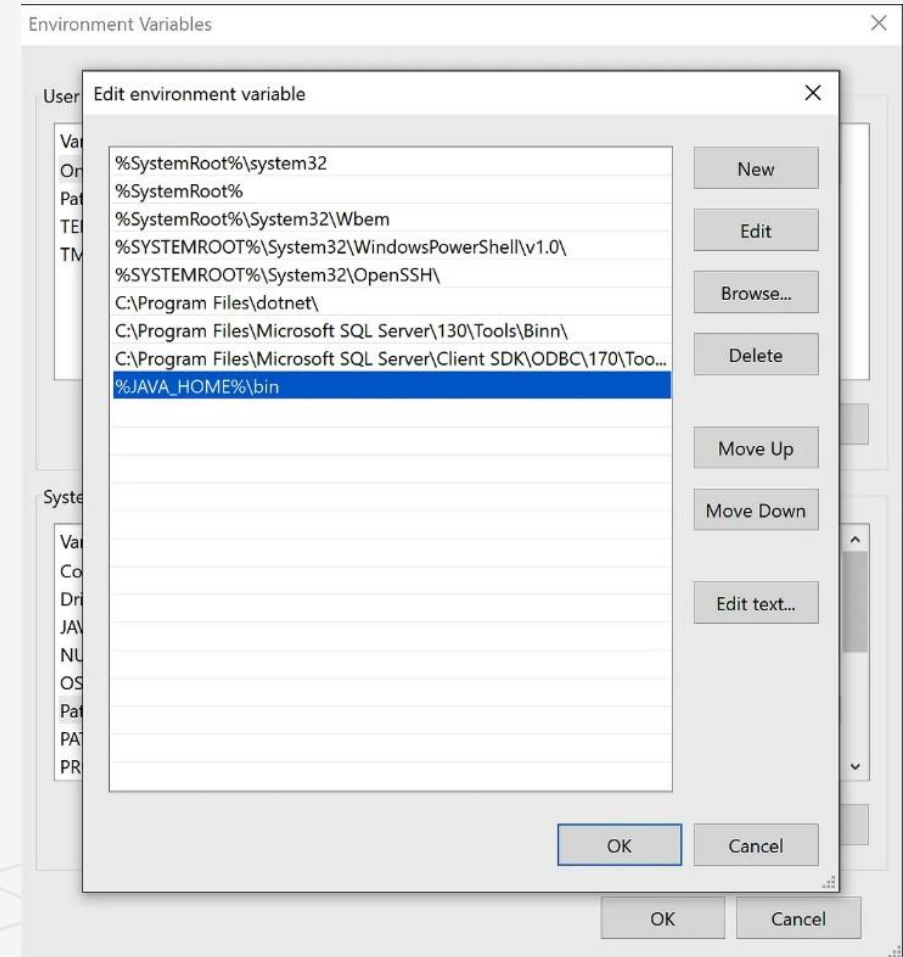
2

Instalasi JDK : Setting path JAVA pada windows

Tambahan variable "JAVA_HOME" di system variable



Edit Path di System Variables, lalu tambahkan lokasi folder bin di JDK



Instalasi JDK : Test Instalasi

1. Buka Command Prompt
2. Ketik "java -version"
3. Jika tampilan sudah seperti dibawah maka berhasil

 Command Prompt

```
Microsoft Windows [Version 10.0.19045.4046]  
(c) Microsoft Corporation. All rights reserved.
```

```
C:\Users\HP>java -version  
java version "17.0.6" 2023-01-17 LTS  
Java(TM) SE Runtime Environment (build 17.0.6+9-LTS-190)  
Java HotSpot(TM) 64-Bit Server VM (build 17.0.6+9-LTS-190, mixed mode, sharing)
```

Instalasi IDE Netbeans

Download IDE Netbeans di [Downloading Apache NetBeans 20](#)

 **Apache NetBeans**

Search the docs

CommunityParticipateBlogGet HelpPluginsDownload

Downloading Apache NetBeans 20

Apache NetBeans 20 was released on December 1, 2023.

Apache NetBeans 20 is available for download from your closest Apache mirror.

Binaries (Platform Independent):

- [netbeans-20-bin.zip](#) (SHA-512, PGP ASC)

Installers and Packages:

- [Apache-NetBeans-20r1-bin-windows-x64.exe](#) (SHA-512, PGP ASC)
- [Apache-NetBeans-20.pkg](#) (SHA-512, PGP ASC)
- [apache-netbeans_20-1_all.deb](#) (SHA-512, PGP ASC)
- [apache-netbeans-20-0.noarch.rpm](#) (SHA-512, PGP ASC)
- [Linux snap package](#)

[Community Installers](#)

[Deployment Platforms](#)

[Known Issues](#)

[Building from Source](#)

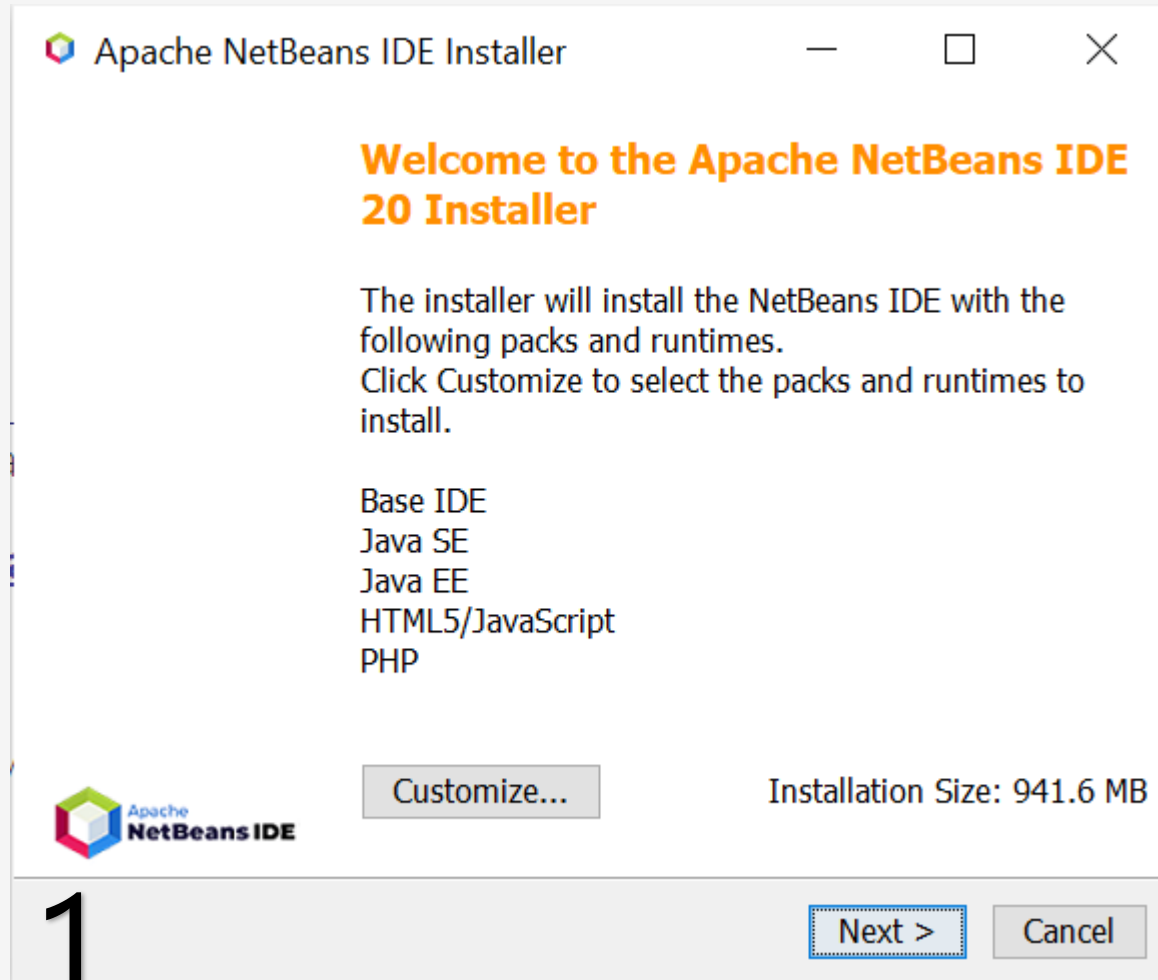
[Community Approval](#)

[Earlier Releases](#)

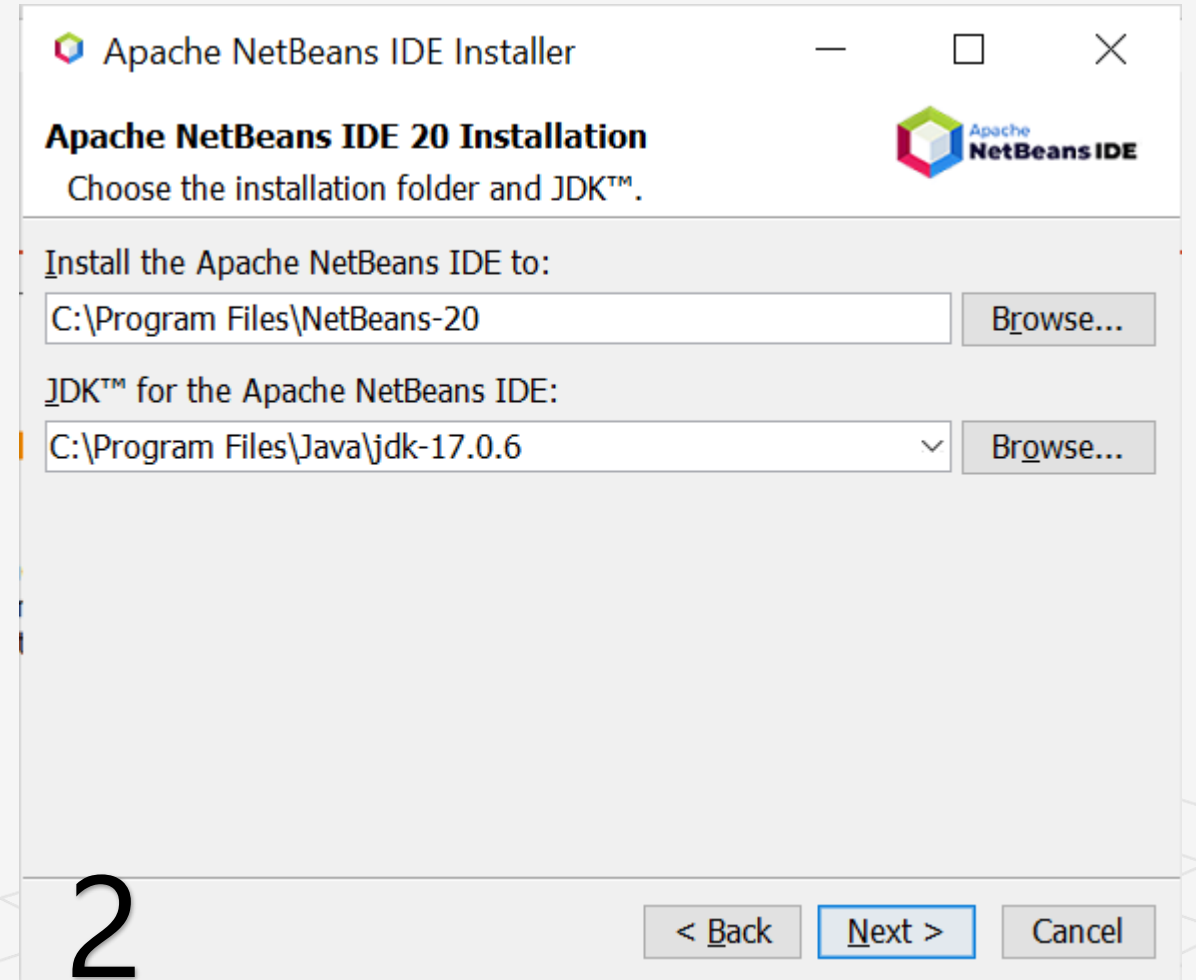


Instalasi IDE Netbeans

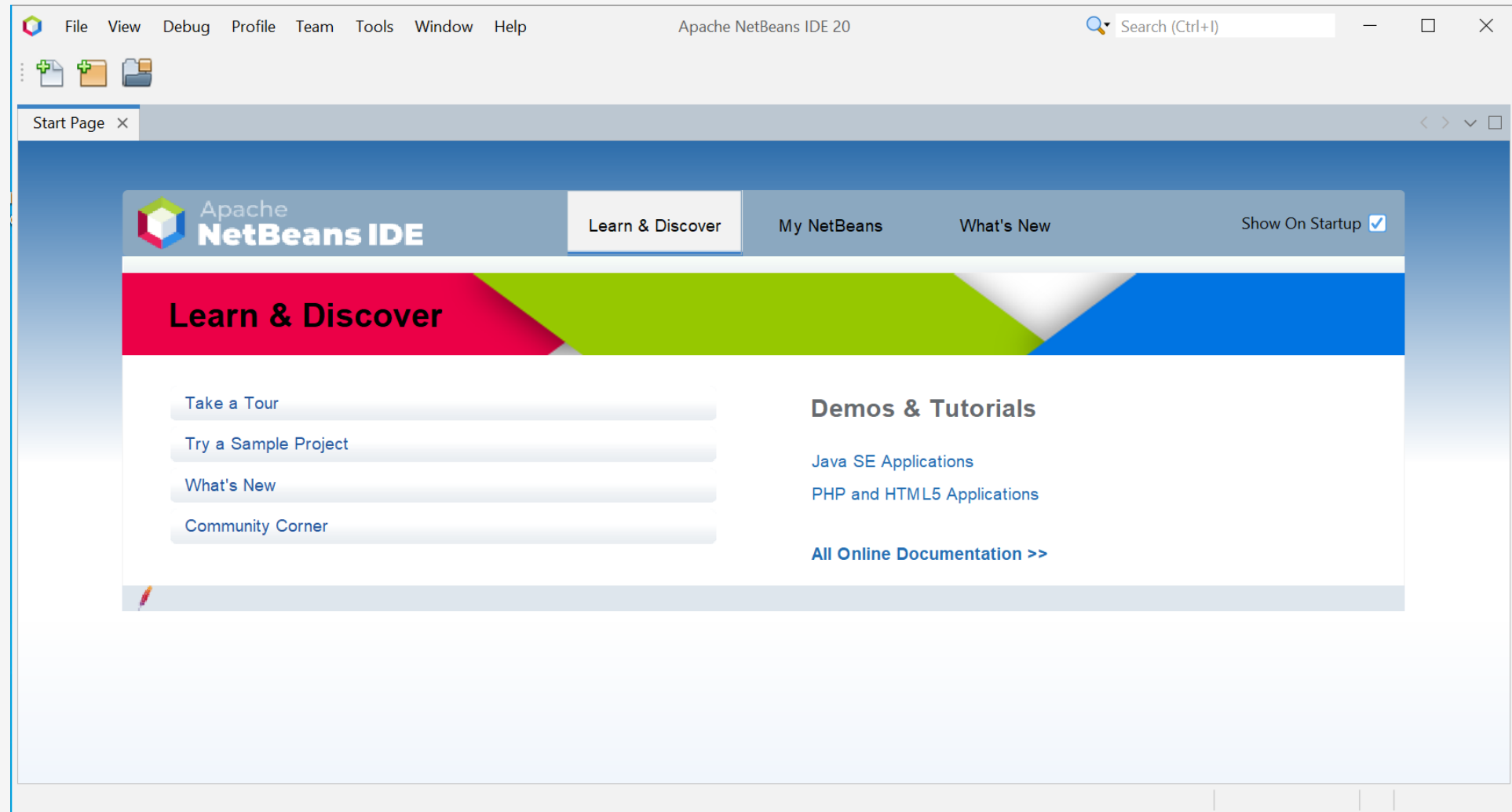
Pilih paket instalasi (biarkan default saja)



Pilih lokasi instalasi dan JDK yang sudah terinstal

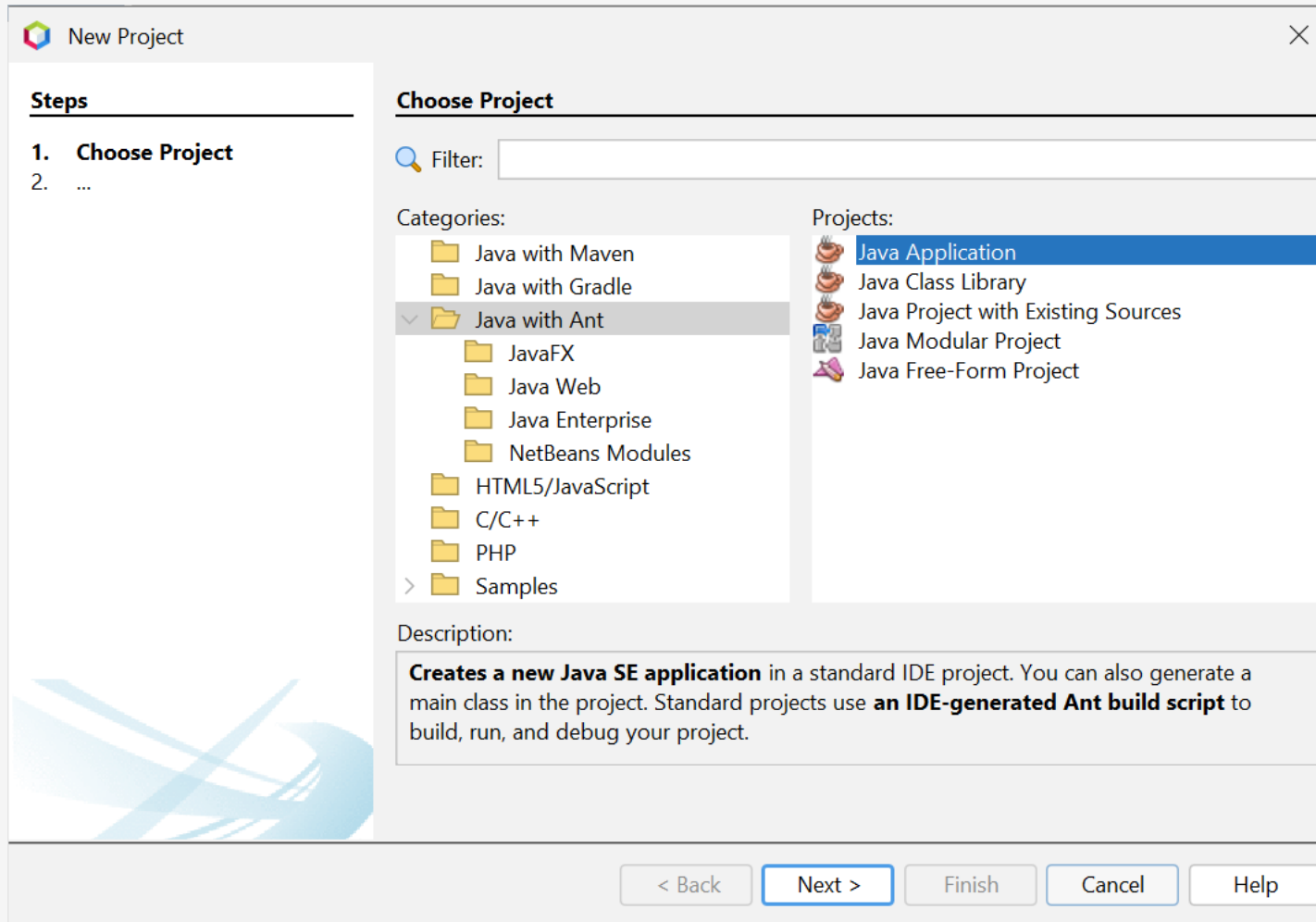


Instalasi IDE Netbeans : Selesai



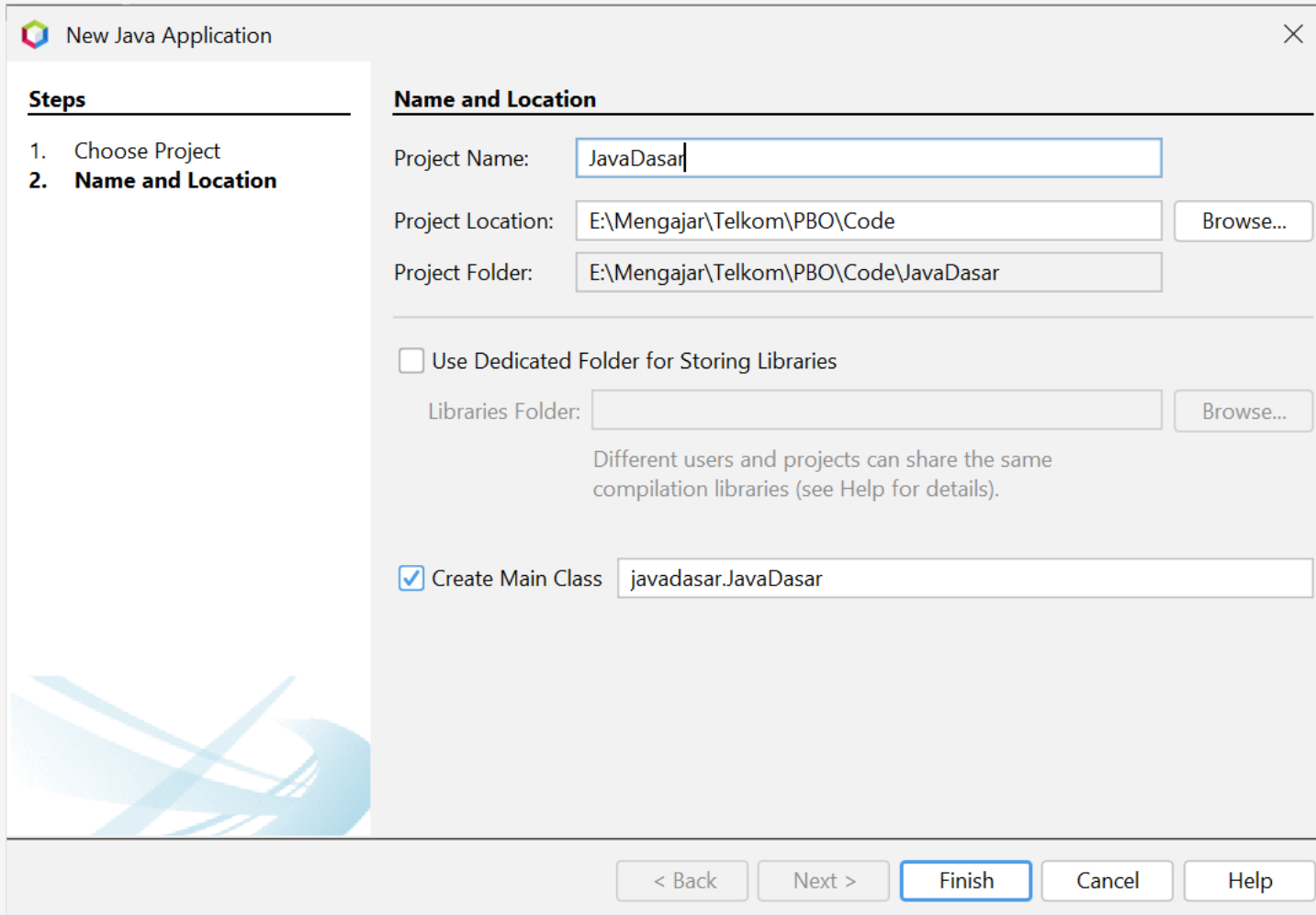
Menulis Code Java pertama Kali : Membuat Project

Klik New Project → Java with Ant → Java Application



Menulis Code Java pertama Kali

Beri nama pada kolom Project Name ➔ Finish



New Java Application

Steps

1. Choose Project
2. **Name and Location**

Name and Location

Project Name:

Project Location: Browse...

Project Folder:

☐ Use Dedicated Folder for Storing Libraries

Libraries Folder: Browse...

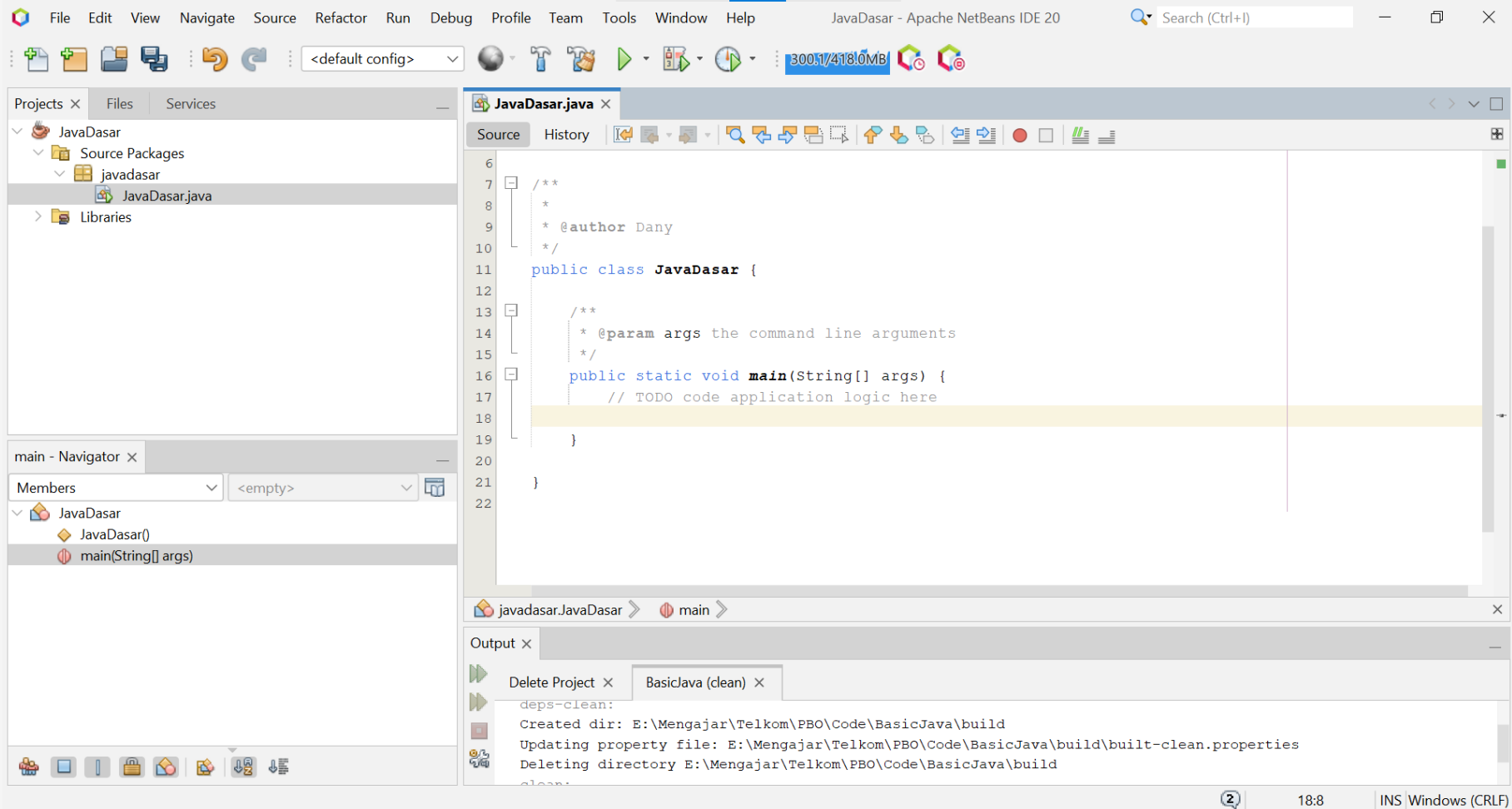
Different users and projects can share the same compilation libraries (see Help for details).

☒ Create Main Class

< Back Next > **Finish** Cancel Help

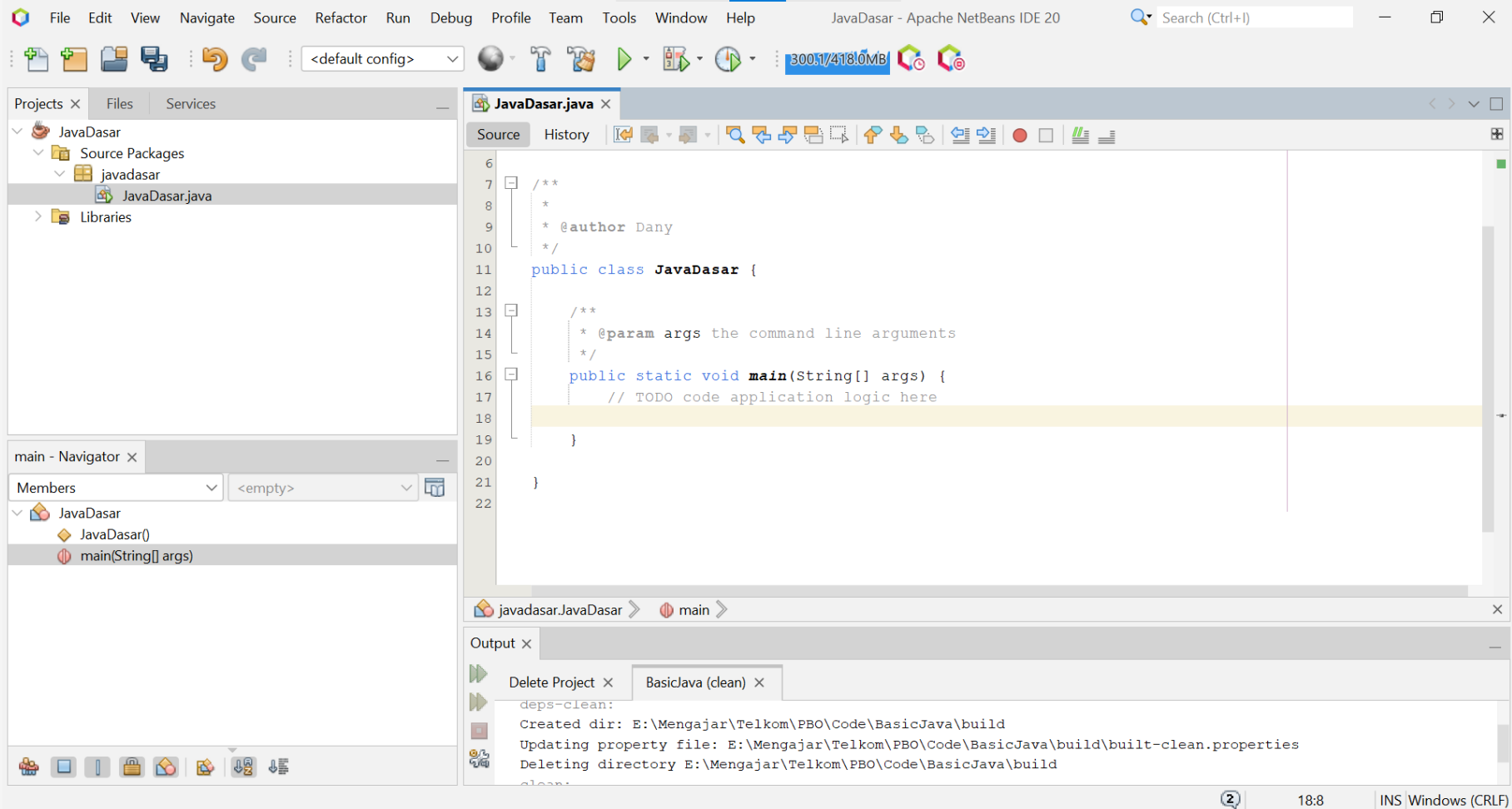
Menulis Code Java pertama Kali

Tampilan lembar kerja project pada IDE Netbeans



Menulis Code Java pertama Kali

Tampilan lembar kerja project pada IDE Netbeans



Menulis Code Java pertama Kali

Tulis Kode yang di HighLight,

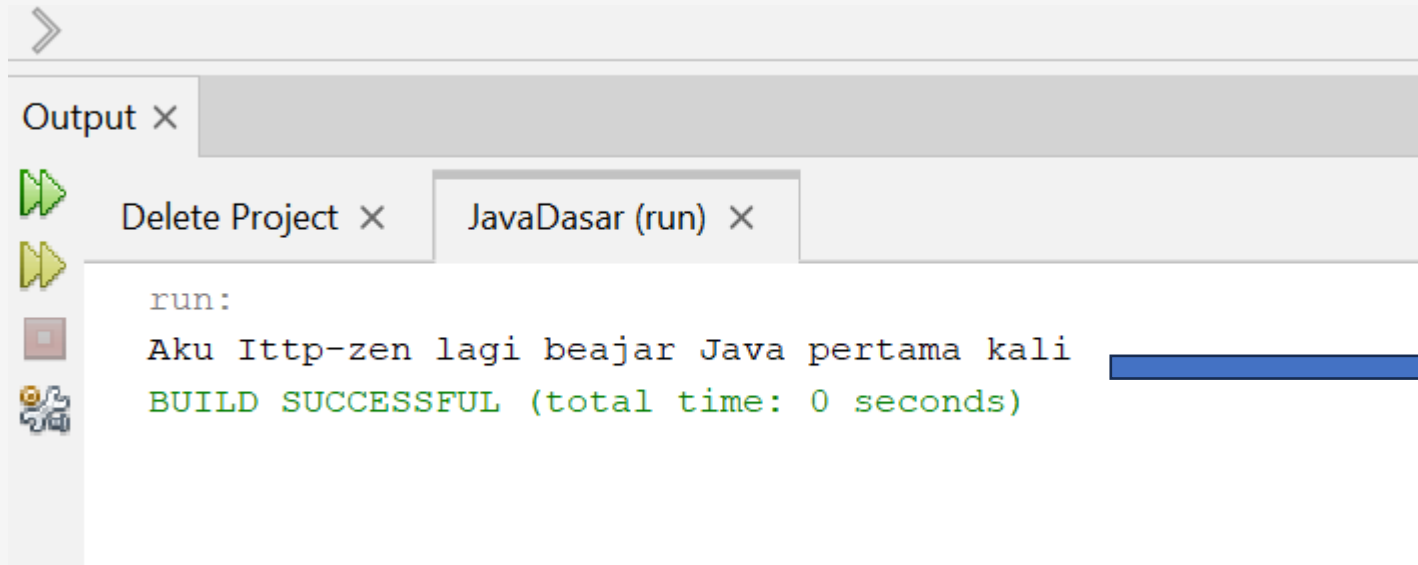
```
^/  
public class JavaDasar {  
  
    /**  
     * @param args the command line arguments  
     */  
    public static void main(String[] args) {  
        // TODO code application logic here  
        System.out.println("Aku Ittp-zen lagi beajar Java pertama kali");  
    }  
}
```

Menulis Code Java pertama Kali: Menjalankan Project

Klik Tombol Run



atau tekan F6



Output Program

Type Data : Tipe Data Bilangan

Tipe Data Bilangan Bulat (Integer Number)

Tipe Data Bilangan Desimal (Floating Point Number)



Tipe Data Bilangan Bulat (Integer Number)

Tipe Data	Nilai Min	Nilai Max	Ukuran pada Memory
byte	-128	127	1 byte
short	-32,768	32,767	2 bytes
int	-2,147,483,648	2,147,483,647	4 bytes
long	-9,223,372,036,854,775,808	9,223,372,036,854,775,807	8 bytes

Contoh Kode Tipe Data Bilangan Bulat (Integer Number)

```
byte tipeByte = 50;
short tipeShort = 1000;
int tipeInt = 21466666;
long tipeLong = 1_000_000_000_000L;

System.out.println("Ini adalah contoh bilangan dengan byte " + tipeByte);
System.out.println("Ini adalah contoh bilangan dengan short " + tipeShort);
System.out.println("Ini adalah contoh bilangan dengan int " + tipeInt);
System.out.println("Ini adalah contoh bilangan dengan long " + tipeLong);
```

Code

Output - JavaDasar (run) X



```
run:
Ini adalah contoh bilangan dengan byte 50
Ini adalah contoh bilangan dengan short 1000
Ini adalah contoh bilangan dengan int 21466666
Ini adalah contoh bilangan dengan long 1000000000000
BUILD SUCCESSFUL (total time: 0 seconds)
```

Hasil

Tipe Data Bilangan Desimal (Floting Point)

Tipe Data	Min	Max	Size
float	3.4e-038	3.4e+038	4 bytes
double	1.7e-308	1.7e+308	8 bytes

Contoh Kode Tipe Data Floating Point Number

```
float tipeFloat1 = 0.13f;
float tipeFloat2 = .25f;

double tipeDouble1 = 43.12;
double tipeDouble2 = 4.0E-43;

System.out.println("Ini adalah tipe data float " + tipeFloat1);
System.out.println("Ini adalah tipe data float " + tipeFloat2);
System.out.println("Ini adalah tipe data double " + tipeDouble1);
System.out.println("Ini adalah tipe data double " + tipeDouble2);
```

Code

Output - JavaDasar (run) ×



run:

Ini adalah tipe data float 0.13

Ini adalah tipe data float 0.25

Ini adalah tipe data double 43.12

Ini adalah tipe data double 4.0E-43

BUILD SUCCESSFUL (total time: 0 seconds)

Hasil

Tipe Data Char

Data Character (huruf) di Java direpresentasikan oleh tipe char.

Pembuatan data char di Java, menggunakan tanda ' (petik satu) di awal dan di akhir karakter

Contoh Kode Tipe Data Char

```
char tipeChar1 = 'a';  
char tipeChar2 = '\n';  
  
System.out.println(tipeChar1);  
System.out.print("Saya"+tipeChar2+"Dany"+tipeChar2);
```

Code

Output - JavaDasar (run) X



run:



a



Saya



Dany

BUILD SUCCESSFUL (total time: 0 seconds)

Hasil

Tipe Data Boolean

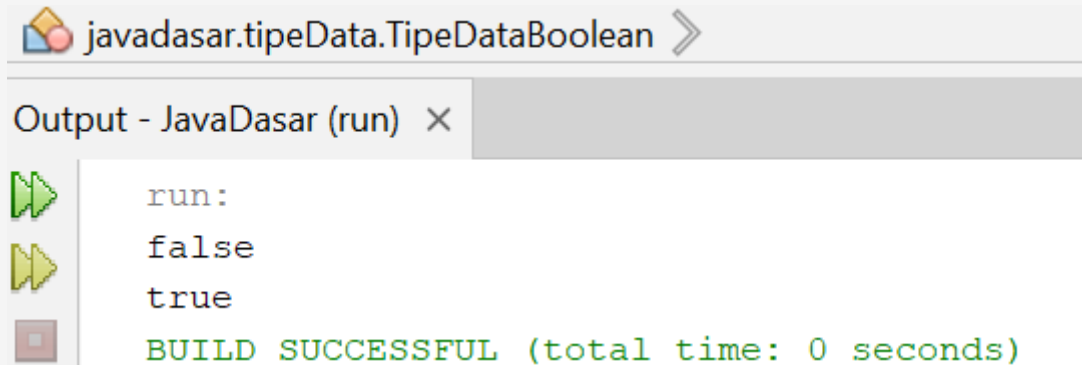
Tipe Data Boolean adalah suatu tipe data logika dengan hanya memiliki 2 nilai yaitu *true* dan *false*.



Contoh Kode Tipe Data Boolean

```
boolean tipeBoolean1=false;  
boolean tipeBoolean2=true;  
  
System.out.println(tipeBoolean1);  
System.out.println(tipeBoolean2);
```

Code



```
javadasar.tipeData.TipeDataBoolean >  
Output - JavaDasar (run) x  
run:  
false  
true  
BUILD SUCCESSFUL (total time: 0 seconds)
```

Hasil

Type Data String

- Tipe data String adalah tipe data yang berisikan data kumpulan karakter atau sederhananya adalah teks
 - Di Java, tipe data String direpresentasikan dengan kata kunci String
 - Pembuatan String di Java, menggunakan karakter " (petik dua) sebelum dan setelah teks nya
 - Default value untuk String adalah null
- 

Code Tipe Data String

```
String nama="Dany";  
String alamat="Purwokerto";  
String institusi="IT Telkom";  
  
System.out.println(nama);  
System.out.println(alamat);  
System.out.println(institusi);
```

Code

Output - JavaDasar (run) X



run:



Dany



Purwokerto

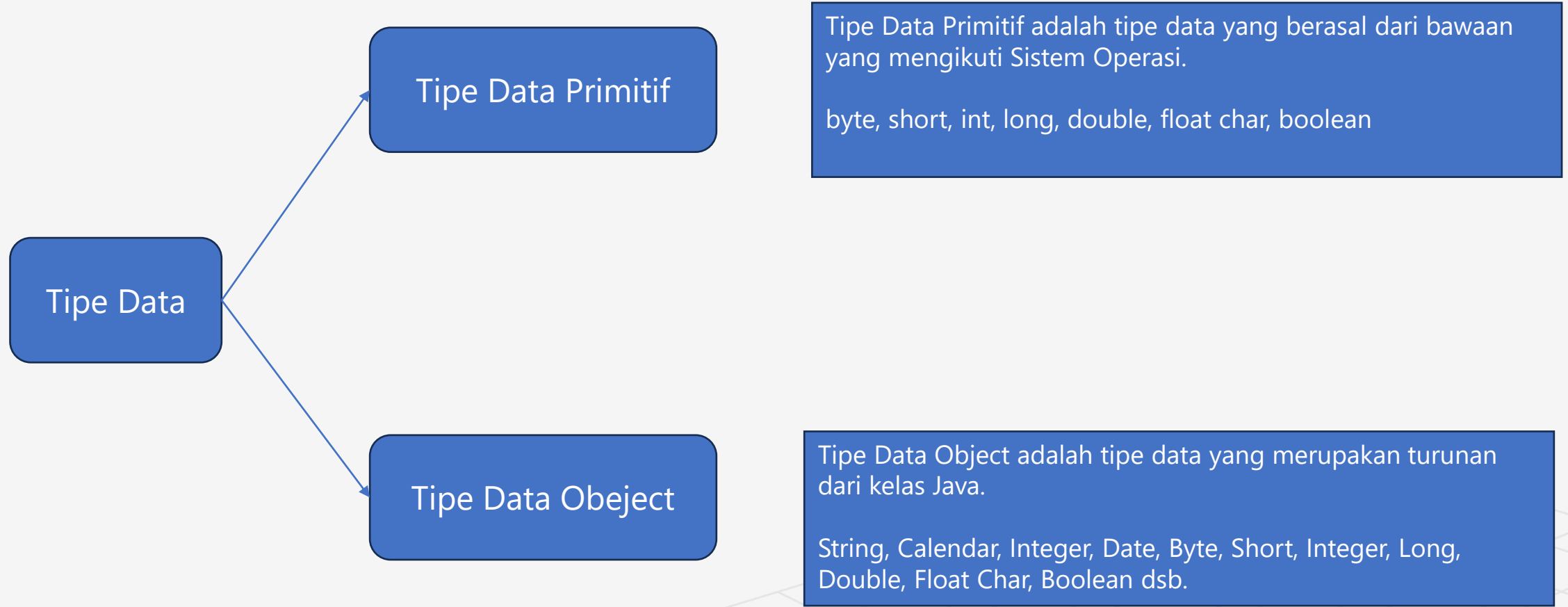


IT Telkom

BUILD SUCCESSFUL (total time: 0 seconds)

Hasil

Tipe Data : Tipe Data pada Java



Variable pada Java

- Variabel merupakan sebuah tempat untuk **menyimpan** data.
- Pada Java setiap pembuatan **variabel harus ditentukan tipe datanya**.
- Cara pembuatan variabel adalah dengan menggunakan **tipe data dan diikuti nama variabel**.

Deklarasi Variable

- Pembuatan variabel harus diawali dengan tipe data dan diikuti dengan nama variabel.
- Nama variabel harus diawali dengan huruf dan diikuti dengan huruf atau angka.
- Nama variabel tidak boleh mengandung huruf unik seperti @, #, \$ dan lain-lain.

Contoh :

```
int nilaiAkhir;  
boolean sudahMenikah
```


Inisialisasi Variable

- Inisialisasi merupakan tahapan menambahkan nilai ke variabel.
- Untuk menambahkan nilai ke variabel, diperlukan perintah sama dengan (=)

Contoh :

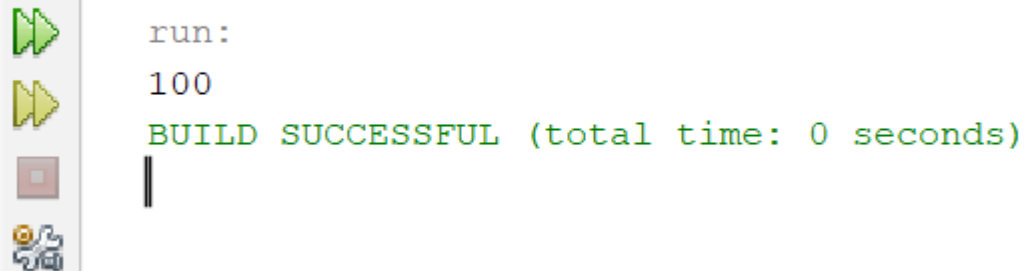
```
int nilaiAkhir = 100;  
boolean sudahMenikah;  
sudahMenikah = true;
```

Contoh Kode Variable

```
int nilaiAkhir;  
  
nilaiAkhir = 100;  
  
System.out.println(nilaiAkhir);  
  
nilaiAkhir = 50;  
  
System.out.println(nilaiAkhir);
```

Code

Output - JavaDasar (run) X



The output window displays the results of running the Java code. It includes a green play button icon, a yellow play button icon, and a red stop button icon. The text shows the output of the program, which is the value 100, followed by a successful build message.

```
run:  
100  
BUILD SUCCESSFUL (total time: 0 seconds)
```

Hasil

Konstanta

- Konstanta merupakan variabel yang tidak dapat diubah lagi nilainya.
- Di Java pembuatan konstanta dapat menggunakan kata kunci final pada awal deklarasi variabel.

Contoh :

```
final int nilaiAkhir = 100;
```

Contoh Code Konstanta

```
final int nilaiAkhir = 100;  
System.out.println(nilaiAkhir);
```

Code

Output - JavaDasar (run) X



run:

100

BUILD SUCCESSFUL (total time: 0 seconds)

|



Hasil

Operator pada Java

- Operator Aritmatika
 - Operator Penugasan
 - Operator Pembandingan
 - Operator Logika
- 
- A decorative grid pattern consisting of light gray lines forming a series of squares, located in the bottom right corner of the slide.

Operator Aritmatika

- Hasil operasi Aritmatika akan mengikuti tipe data variable yang dibuat
- Variable bertipe int akan menghasilkan int

Operator	Keterangan
+	Tambah
-	Kurang
*	Kali
/	Bagi
%	Sisa Bagi

Code Operator Aritmatika

```
int a = 5;
int b = 25;

int hasilKali = a * b;
System.out.println(hasilKali);

int hasilKurang = b - a;
System.out.println(hasilKurang);

int hasilTambah = a + b;
System.out.println(hasilTambah);

int hasilBagi = b / a;
System.out.println(hasilBagi);

int hasilModulus = b % a;
System.out.println(hasilModulus);
```

Code

Output - JavaDasar (run) X



run:

125

20

30

5

0

BUILD SUCCESSFUL (total time: 0s)

Hasil

Operator Unary

Operator Unary adalah operator yang ditempatkan didepan variable untuk menegaskan sesuatu.

Operator	Keterangan
+	Positif
-	Negatif
++	Tambah 1
--	Kurang 1
!	Kebalikan nilai boolean

Code Operator Unary

```
int a = 10;  
  
System.out.println(--a);  
  
int b = 10;|  
  
System.out.println(++b);
```

Code

Output - JavaDasar (run) ×

```
run:  
9  
11  
BUILD SUCCESSFUL (total time: 0 seconds)
```

Hasil

Operator Penugasan

- Operator penugasan berguna untuk memberi **nilai ke suatu variabel**

Operator	Keterangan
=	Pemberian Nilai
+=	Penambahan Nilai
-=	Pengurangan Nilai
*=	Perkalian Nilai
/=	Pembagian Nilai
%=	Sisa Pembagian Nilai

Expression	Meaning
$x += y$	$x = x + y$
$x -= y$	$x = x - y$
$x *= y$	$x = x * y$
$x /= y$	$x = x / y$

Contoh Code Operator Penugasan

```
int a = 10;  
  
System.out.println(a);  
  
a += 5;  
  
System.out.println(a);  
  
a += 5;  
  
System.out.println(a);
```

Code

Output - JavaDasar (run) X



run:



10



15



20

BUILD SUCCESSFUL (total time: 0 seconds)

Hasil

Operator Pembanding

- Operasi perbandingan adalah operasi untuk membandingkan dua buah data
- Operasi perbandingan adalah operasi yang menghasilkan nilai boolean (benar atau salah)
- Jika hasil operasinya adalah benar, maka nilainya adalah true
- Jika hasil operasinya adalah salah, maka nilainya adalah false

Operator	Keterangan
=	Pemberian Nilai
+=	Penambahan Nilai
-=	Pengurangan Nilai
*=	Perkalian Nilai
/=	Pembagian Nilai
%=	Sisa Pembagian Nilai

Contoh Code Operator Pembandingan

```
int a = 4;  
int b = 13;  
  
boolean c = a < b;  
  
System.out.println(c);
```

Code

Output - JavaDasar (run) X



```
run:  
true  
BUILD SUCCESSFUL (total time
```

Hasil

Operator Logika

Operator logika menggunakan ekspresi dengan nilai balik Boolean (true or false)

Operator	Keterangan
&&	Dan
	Atau

Hasil Operator Logika

Nilai 1	Operator	Nilai 2	Hasil
true	&&	true	true
true	&&	false	false
false	&&	true	false
false	&&	false	false

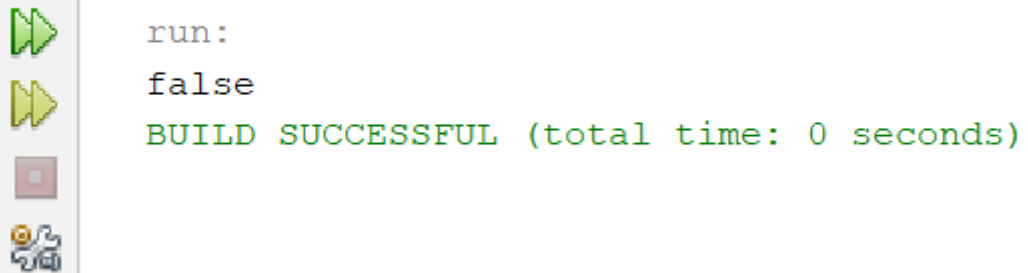
Nilai 1	Operator	Nilai 2	Hasil
true		true	true
true		false	true
false		true	true
false		false	false

Contoh Code Operator Logika

```
int nilai = 80;  
int absensi = 80;  
int tugas = 75;  
  
boolean hasil = (nilai > 75 || absensi > 75) && tugas > 75;  
  
System.out.println(hasil);
```

Code

Output - JavaDasar (run) X



The output window displays the results of running the Java code. It includes a green play button icon, a yellow play button icon, a red stop button icon, and a gear icon. The text shows the variable 'hasil' is assigned the value 'false', followed by a success message.

```
run:  
false  
BUILD SUCCESSFUL (total time: 0 seconds)
```

Hasil

Output

Untuk menampilkan tulisan pada terminal / console dapat menggunakan perintah :

Ex:

System.out.print(...);

System.out.println(...);

A decorative graphic in the bottom right corner of the slide, consisting of a grid of diamond shapes (squares rotated 45 degrees) in a light gray color, creating a perspective effect.

Contoh Code Output

```
System.out.print("Dany Candra\n");  
System.out.println("Dany \nCandra ");
```

Code

Output - JavaDasar (run) ×



run:



Dany Candra



Dany



Candra

BUILD SUCCESSFUL (total time: 0 seconds)

Hasil

Input

Input di terminal / console di Java dapat menggunakan kelas **java.util.Scanner**

Ex:

```
import java.util.Scanner;
```

```
Scanner input = new Scanner(System.in);
```

A decorative grid pattern consisting of light gray lines forming a series of squares, located in the bottom right corner of the slide.

Function pada kelas scanner

Menggunakan class **Scanner** (java.util.Scanner) dengan method sebagai berikut:

1. **nextInt()** : untuk menerima tipe data integer
2. **nextShort()** : untuk menerima tipe data short
3. **nextLong()** : untuk menerima tipe data long
4. **nextDouble()** : untuk menerima tipe data double
5. **nextFloat()**: untuk menerima tipe data float
6. **nextLine()** : untuk menerima tipe data string
7. **nextBoolean()** : untuk menerima tipe data boolean

Contoh Code

```
Scanner input = new Scanner(System.in);

System.out.print("Nilai : ");
int nilai = input.nextInt();

input.nextLine();
System.out.print("Nama : ");
String nama = input.nextLine();

System.out.print("Menikah : ");
boolean menikah = input.nextBoolean();

System.out.println("Nilai yang dimasukkan adalah " + nilai);
System.out.println("Nama yang dimasukkan adalah " + nama);
System.out.println("Menikah yang dimasukkan adalah " + menikah);
```

Output - JavaDasar (run) X



run:

Nilai : 80

Nama : Dany

Menikah : false

Nilai yang dimasukkan adalah 80

Nama yang dimasukkan adalah Dany

Menikah yang dimasukkan adalah false

Code

Hasil

Array

Array merupakan kumpulan data pada sebuah variabel dengan tipe data yang sama.

Pembuatan array menggunakan tanda []

Membuat Array

Perintah pembuatan Array :

tipedata[] variabel = new typedata[jumlah];

Contoh :

int[] nilai = new nilai[10];
String[] nama = new String[10];



Memanipulasi Array

Untuk mengubah dan mengambil nilai Array, dapat menggunakan indeks.

Indeks Array dimulai dari 0 sampai (jumlah – 1)

```
String[] nama = new String[3];  
nama[0] = "Dany";  
nama[1] = "Candra";  
nama[2] = "Febrianto";
```

Inisialisasi Array secara Langsung

Array dapat diinisialisasi secara langsung saat proses pendeklarasian.
Contoh :

```
String[] nama = {  
    "Dany", "Candra", "Febrianto"  
};
```


Contoh Code Array dan Manipulasi Array

```
//deklarasi array
int[] nilaiKelas = new int[5];
//mengisi nilai array
nilaiKelas[0] = 100;
nilaiKelas[1] = 85;
nilaiKelas[2] = 75;
nilaiKelas[3] = 88;
nilaiKelas[4] = 90;

System.out.println(nilaiKelas[3]);

// mengganti nilai dalam array
nilaiKelas[3] = 80;

System.out.println(nilaiKelas[3]);
```

Code

Output - JavaDasar (run) X



run:

88

80

BUILD SUCCESSFUL (total time: 0 seconds)

Hasil

Contoh Code Deklarasi Array secara langsung

```
String[] namaSiswa = {  
    "Dany", "Candra", "Febrianto"  
};  
  
System.out.println(namaSiswa[0]);  
System.out.println(namaSiswa[1]);  
System.out.println(namaSiswa[2]);  
  
int panjang = namaSiswa.length;  
System.out.println(panjang);
```

Code

Output - JavaDasar (run) X



run:

Dany

Candra

Febrianto

3

BUILD SUCCESSFUL (total time: 0

Hasil

Array Multidimensi

- Array multidimensi merupakan Array yang terdapat di dalam Array.
- Array multidimensi dapat dibuat dengan menambahkan tanda [] sesuai dengan jumlah yang akan dibuat.



Membuat Array Multidimensi

Perintah pembuatan Array Multidimensi :

```
tipe[][] var = new tipe[jumlah][jumlah];
```

Contoh :

```
int[][] contoh1 = new int[10][3];
```

```
String[][] contoh2 = new String[5][5];
```



Manipulasi Array Multidimensi

Untuk memanipulasi data pada Array multidimensi, sama seperti Array biasa.

```
String[][] contoh = new String[3][2]
```

```
contoh[0][0] = "Dany";  
contoh[0][1] = "Candra";
```

```
contoh[1][0] = "Adit";  
contoh[1][1] = "Rully";
```

```
contoh[2][0] = "Dimas";  
contoh[2][1] = "Farkhan";
```



Inisialisasi Secara Langsung

```
String[][] nama = {  
    { "Dany", "Candra" },  
    { "Joko", "Moro" },  
    { "Aditya", "Yudha" },  
    { "Rully", "Hidayat" }  
};
```

Contoh Code Array Multidimensi

```
int[][] nilai = new int[3][2];
```

```
nilai[0][0] = 10;
```

```
nilai[0][1] = 10;
```

```
nilai[1][0] = 7;
```

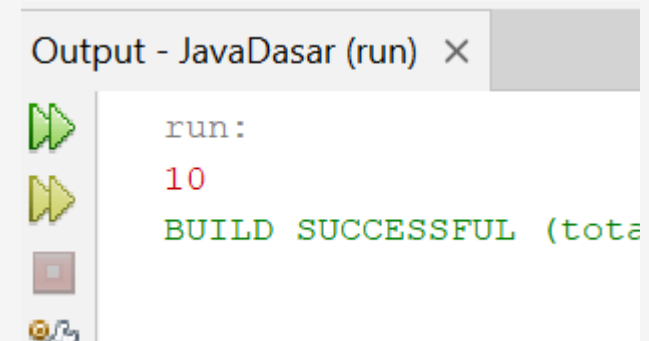
```
nilai[1][1] = 8;
```

```
nilai[2][0] = 8;
```

```
nilai[2][1] = 8;
```

```
System.out.println(nilai[0][0]);
```

Code

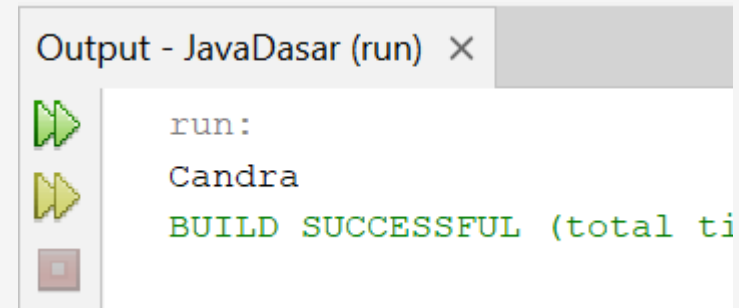


Hasil

Contoh Code Array Multidimensi

```
String[][] nama = {  
    { "Dany", "Candra" },  
    { "Joko", "Moro" },  
    { "Aditya", "Pradana" },  
    { "Rully", "Hidayat" }  
};  
  
System.out.println(nama[0][1]);
```

Code



Output - JavaDasar (run) ×

run:
Candra
BUILD SUCCESSFUL (total ti

Hasil

Blok

- Sebelum belajar percabangan, Anda harus terlebih dahulu mengenal blok.
- Blok merupakan area kode yang diapit oleh kurung kurawal { isi blok }.
- Blok dapat dibuat didalam blok lain.
- Sebuah variabel di blok terluar, dapat diakses dari blok didalamnya.
- Namun sebaliknya, variabel di blok dalam, **tidak dapat** diakses dari blok luarnya.

Contoh Blok

```
public static void main(String[] args) { // blok 0  
  
    String nama = "Dany";  
  
    // blok 1  
    {  
        System.out.println(nama);  
  
        // blok 1.1  
        {  
            System.out.println(nama);  
        }  
    }  
}
```

Code

Output - JavaDasar (run) X



run:

Dany

Dany

BUILD SUCCESSFUL (total time: 0 seconds)

Hasil

Percabangan IF statement

- Dalam Java, if adalah salah satu kata kunci yang digunakan untuk percabangan
- Percabangan artinya kita bisa mengeksekusi kode program tertentu ketika suatu kondisi terpenuhi
- Hampir di semua bahasa pemrograman mendukung if expression

Contoh :

```
if(kondisi){  
    // isi perintah jika kondisi benar  
}
```

Contoh

```
int nilai = 90;
int tugas = 85;

if(nilai > 80 && tugas > 80){
    // perintah
    System.out.println("Selamat Nilai dan Tugas Anda Bagus!!!");
}
```

Code

Output - JavaDasar (run) ×



run:

Selamat Nilai dan Tugas Anda Bagus!!!

BUILD SUCCESSFUL (total time: 0 seconds)

Hasil

Percabangan Else Statement

- Blok if akan dieksekusi ketika kondisi if bernilai true
- Kadang kita ingin melakukan eksekusi program tertentu jika kondisi if bernilai false
- Hal ini bisa dilakukan menggunakan else expression

Contoh :

```
if(kondisi){  
    // isi perintah jika kondisi benar  
}else{  
    // isi perintah jika kondisi salah  
}
```

Contoh

```
int nilai = 75;

if(nilai > 80){
    // perintah jika kondisi true
    System.out.println("Selamat!!!");
}else{
    // perintah jika kondisi false
    System.out.println("Coba Lagi!!!");
}
```

Code

Output - JavaDasar (run) X



```
run:
Coba Lagi!!!
BUILD SUCCESSFUL (total time: 0 seconds)
```

Hasil

Percabangan Else If (Bersarang)

- Kada dalam If, kita butuh membuat beberapa kondisi
- Kasus seperti ini, di Java kita bisa menggunakan Else If expression
- Else if di Java bisa lebih dari satu

```
if(kondisi1){  
    // isi perintah jika kondisi 1 benar  
}else if(kondisi2){  
    // isi perintah jika kondisi 2 salah  
} else if(...){  
    // .....  
}else{  
    // isi perintah jika semua kondisi salah  
}
```

Contoh

```
int nilai = 75;
int tugas = 75;

if(nilai >= 80 && tugas >= 80){
    System.out.println("Selamat Nilai dan Tugas Anda Bagus!!!");
}else if(nilai >= 80){
    System.out.println("Selamat Nilai Anda Bagus!!!");
}else if(tugas >= 80){
    System.out.println("Selamat Tugas Anda Bagus!!!");
}else{
    System.out.println("Coba Lagi!!!");
}
```

Code

Output - JavaDasar (run) ×



run:

Coba Lagi!!!

BUILD SUCCESSFUL (total time: 0 seconds)

Hasil

Switch Statement

- Kadang kita hanya butuh menggunakan kondisi sederhana di if statement, seperti hanya menggunakan perbandingan ==
- Switch adalah statement percabangan yang sama dengan if, namun lebih sederhana cara pembuatannya
- Kondisi di switch statement hanya untuk perbandingan ==

```
switch(variabel){  
    case 1 :  
        // isi perintah jika variabel = 1  
        break;  
    case 2:  
    case 3:  
        // isi perintah jika variabel = 2 atau 3  
        break;  
    default :  
        // isi perintah jika variabel sesuai  
        break;  
}
```

Contoh

```
int nomor = 2;

switch(nomor) {
    // nomor == 1
    case 1 :
        System.out.println("Satu");
        break;
    case 2 :
        System.out.println("Dua");
        break;
    case 3 :
        System.out.println("Tiga");
        break;
    case 4 :
        System.out.println("Empat");
        break;
    default :
        System.out.println("Bukan Satu Sampai Empat");
        break;
}
```

Code

Output - JavaDasar (run)

```
run:
Dua
BUILD SUCCESSFUL (total time: 0 seconds)
```

Hasil

Perulangan For Loop

- For adalah salah satu kata kunci yang bisa digunakan untuk melakukan perulangan
- Blok kode yang terdapat di dalam for akan selalu diulangi selama kondisi for terpenuhi

```
for (inisialisasi; kondisi; increment) {  
    // lakukan selama kondisi benar  
}
```

Contoh

```
for( int i = 1 ; i <= 100 ; i++ ){  
    System.out.println(i);  
}
```

Code

Output - JavaDasar (run) X



96



97



98



99

100

BUILD SUCCESSFUL (total time: 0 seconds)

Hasil

While Loop

- While loop adalah versi perulangan yang lebih sederhana dibanding for loop
- Di while loop, hanya terdapat kondisi perulangan, tanpa ada init statement dan post statement

```
while( kondisi ){  
    // lakukan selama kondisi benar  
}
```

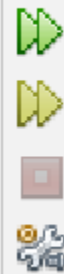
Contoh

```
int i = 1;

while(i <= 100){
    System.out.println(i);
    i++;
}
```

Code

Output - JavaDasar (run) X



96
97
98
99
100

Output - JavaDasar (run)

BUILD SUCCESSFUL (total time: 0 seconds)

Hasil

Do-While Loop

- Do While loop adalah perulangan yang mirip dengan while
- Perbedaannya hanya pada pengecekan kondisi
- Pengecekan kondisi di while loop dilakukan di awal sebelum perulangan dilakukan, sedangkan di do while loop dilakukan setelah perulangan dilakukan
- Oleh karena itu dalam do while loop, minimal pasti sekali perulangan dilakukan walaupun kondisi tidak bernilai true

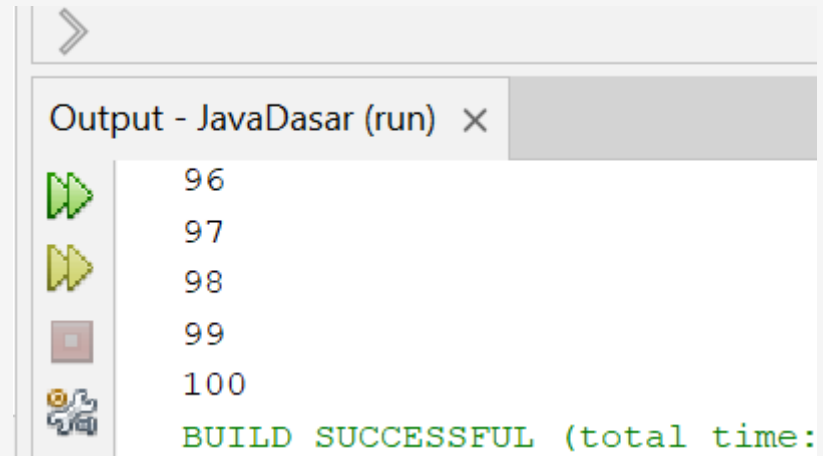
```
do {  
    // lakukan selama kondisi benar  
} while ( kondisi );
```

Contoh

```
int i = 1;

do {
    System.out.println(i);
    i++;
} while (i <= 100);
```

Code



```
Output - JavaDasar (run) X
96
97
98
99
100
BUILD SUCCESSFUL (total time:
```

Hasil

Continue

- Perintah continue digunakan untuk men-skip kode program selanjutnya.
- Dalam perulangan continue bisa digunakan sebagai perintah untuk melanjutkan ke perulangan berikutnya, tanpa harus mengeksekusi kode program.

continue;

A decorative graphic in the bottom right corner consisting of a grid of thin, light gray lines forming a series of overlapping diamond shapes, creating a perspective effect.

Contoh

```
for(int i = 1; i <= 100; i++){  
  
    if(i % 2 == 1){  
        continue;  
    }  
  
    System.out.println(i+" adalah bilangan genap");  
}
```

Code

Output - JavaDasar (run) X

92 adalah bilangan genap
94 adalah bilangan genap
96 adalah bilangan genap
98 adalah bilangan genap
100 adalah bilangan genap
BUILD SUCCESSFUL (total time: 0 se

Hasil

Break

- Perintah break digunakan untuk menghentikan proses perulangan.
- Dengan menggunakan perintah break, sisa perulangan selanjutnya tidak akan dieksekusi lagi.

break;

A decorative graphic in the bottom right corner consisting of a grid of thin, light gray lines forming a series of overlapping diamond shapes, creating a perspective effect.

Contoh

```
for(int i = 1; i <= 100; i++){  
    if(i == 5){  
        break;  
    }  
    System.out.println(i);  
}
```

Code

Output - JavaDasar (run) X



run:



1



2



3

4

BUILD SUCCESSFUL (total time: 0 seconds)

Hasil

Perulangan For-Each

- Kadang kita biasa mengakses data array menggunakan perulangan
- Mengakses data array menggunakan perulangan sangat bertele-tele, kita harus membuat counter, lalu mengakses array menggunakan counter yang kita buat
- Namun untungnya, di Java terdapat perulangan for each, yang bisa digunakan untuk mengakses seluruh data di Array secara otomatis

```
for( typedata variabel : array ){  
    // variabel adalah data yang ada di array  
}
```

Contoh

```
String[] nama = {  
    "Dany", "Joko", "Adit", "Ruly", "Budi", "Chandra"  
};  
  
for(String hasil : nama){  
    System.out.println(hasil);  
}
```

Code

Output - JavaDasar (run) ×



Joko



Adit



Ruly



Budi

Chandra

BUILD SUCCESSFUL (total time: 0 sec)

Hasil

Source Code Lengkap

https://drive.google.com/file/d/17Mi9aKpT_wk_4Mu_YhRveEGhtAqnV8-c/view?usp=sharing

Materi Selanjutnya

OOP

Object dan Class

Atribut

Konstanta

Prosedur

Fungsi

Keyword This

Konstruktur

Paket

Visibility

Encapsulation

